KYAMBOGO UNIVERSITY

FACULTY OF ENGINEERING · DEPARTMENT OF ELECTRICAL AND ELECTRONICS

TETE 3102: MICROCONTROLLERS AND APPLICATIONS

LABORATORY REPORT: TASK 2

I2C Bus Scanner on Dual Platforms: STM32 Nucleo & Arduino Uno

Mukama Robert (26/U/ETE/21285/PE)
Nabudere Zerubabel (25/U/ETE/05465/PE)
Nanfuka Shakirah (25/U/ETE/05468/PE)
Kidega Wilkie Cephas (25/U/ETE/05460/PE)

Group Number: Group 4
Platforms: STM32F411RE & Arduino Uno
Course Supervisor: Dr. Mugerwa Dickson PhD

OBJECTIVE & LAB FOCUS

The objective of this laboratory investigation is to design, deploy, and evaluate an I^2C bus scanner on both the 8-bit AVR architecture (Arduino Uno) and the 32-bit ARM Cortex-M4 architecture (STM32F411RE). Students verify device presence across the 7-bit address spectrum, analyze physical bus disconnection faults, and explain address filtering for reserved I^2C protocol ranges.

1 Hardware Setup & Interfacing Topology

The experimental bench comprises an STM32 Nucleo-F411RE board, an Arduino Uno R3, an HD44780 LCD with PCF8574 I^2C backpack, and a DS3231 High-Precision Real-Time Clock (RTC) module.

Table 1: System Pinout and Wiring Interconnections

Peripheral	Pin Function	Arduino Uno Pin	STM32 Nucleo Pin	Electrical Specification
LCD1602 (PCF8574)	VCC	5V	5V (from Nucleo)	Supply rail for LCD contrast
	GND	GND	GND	Common ground reference
	SDA	A4	PB9 (D14)	AF4 Open-Drain
	SCL	A5	PB8 (D15)	AF4 Open-Drain
DS3231 RTC	VCC	3.3V or 5V	3.3V	2.3V to 5.5V supply
	GND	GND	GND	Common ground reference
	SDA	A4	PB9	Shared I2C serial data
	SCL	A5	PB8	Shared I2C serial clock
Pull-Up Resistors	R_p (4.7 kΩ)	A4, A5 to 5V	PB8, PB9 to 3.3V	Ensures fast rise time

HARDWARE & PROTOCOL TROUBLESHOOTING NOTE

Critical Voltage Tolerance Note: While STM32 pins PB8 and PB9 are Five-Volt Tolerant (FT), external pull-up resistors on a bus shared with a 3.3V microcontroller must be pulled to **3.3V**, or a bidirectional level shifter must be inserted if 5V pull-ups on breakout boards cannot be disabled.

2 Firmware Implementation

2.1 Platform 1: Arduino Wire Bus Scanner

The Arduino scanner interrogates each 7-bit address by initiating a write frame (`Wire.beginTransmission(addr)`) and ending it (`Wire.endTransmission()`). If the slave responds with an ACK, `endTransmission()` returns `0`.

Listing 1: Arduino Uno I2C Bus Scanner Implementation

```
#include <Wire.h>

void setup() {
  Serial.begin(115200);
  while (!Serial); // Wait for serial monitor connection
  Wire.begin();
  Serial.println("\n--- Kyambogo University TETE 3102 ---");
  Serial.println("Arduino Wire I2C Bus Scanner Initialized");
}

void loop() {
  byte error, address;
  int nDevices = 0;

  Serial.println("\nScanning bus addresses (0x08 to 0x77)...");
  for (address = 8; address < 120; address++) {
    Wire.beginTransmission(address);
    error = Wire.endTransmission();

    if (error == 0) {
      Serial.print("SUCCESS: Device found at 7-bit address 0x");
      if (address < 16) Serial.print("0");
      Serial.print(address, HEX);
      Serial.print("  [Write: 0x");
      Serial.print((address << 1), HEX);
      Serial.print(", Read: 0x");
      Serial.print((address << 1) | 1, HEX);
      Serial.println("]");
      nDevices++;
    } else if (error == 4) {
      Serial.print("ERROR: Unknown protocol error at address 0x");
      if (address < 16) Serial.print("0");
      Serial.println(address, HEX);
    }
  }

  if (nDevices == 0) {
    Serial.println("No I2C devices responded on the bus.");
  } else {
    Serial.print("Bus scan complete. Found ");
    Serial.print(nDevices);
    Serial.println(" active device(s).");
  }

  delay(4000); // Repeat scan every 4 seconds
}
```

2.2 Platform 2: STM32F411RE HAL Bus Scanner

In the STM32 HAL, bus discovery is performed using `HAL_I2C_IsDeviceReady()`. This function transmits a START condition and the 8-bit shifted address, waiting up to `timeout` milliseconds for the slave's ACK bit.

Listing 2: STM32 HAL I2C Bus Scanner Implementation (main.c)

```c
/* Includes placed in USER CODE BEGIN Includes */
#include <stdio.h>
#include <string.h>

/* Code placed in USER CODE BEGIN 2 */
char msg[80];
int len;

len = snprintf(msg, sizeof(msg), "\r\n=== STM32F411 I2C Scanner (HAL_I2C_IsDeviceReady) ===\r\n");
HAL_UART_Transmit(&huart2, (uint8_t*)msg, len, 100);

uint8_t count = 0;
for (uint8_t addr7 = 8; addr7 < 120; addr7++) {
    /* Pass the 7-bit address shifted left by 1 bit into HAL */
    uint16_t wire_addr = (uint16_t)(addr7 << 1);

    /* Trials = 2, Timeout = 10 ms */
    if (HAL_I2C_IsDeviceReady(&hi2c1, wire_addr, 2, 10) == HAL_OK) {
        len = snprintf(msg, sizeof(msg),
                       "Found Device: 7-bit: 0x%02X | Write: 0x%02X | Read: 0x%02X\r\n",
                       addr7, wire_addr, wire_addr | 0x01);
        HAL_UART_Transmit(&huart2, (uint8_t*)msg, len, 100);
        count++;
    }
}

if (count == 0) {
    len = snprintf(msg, sizeof(msg), "No devices responded on I2C1.\r\n");
    HAL_UART_Transmit(&huart2, (uint8_t*)msg, len, 100);
} else {
    len = snprintf(msg, sizeof(msg), "Scan finished. Total devices detected: %d\r\n",
        count);
    HAL_UART_Transmit(&huart2, (uint8_t*)msg, len, 100);
}
/* USER CODE END 2 */
```

3 Experimental Observations & Fault Injections

4 Mandatory Task Questions & Detailed Solutions

Task Question 1

Do the Arduino and STM32 give the same list of detected addresses?

Answer: Yes, both platforms return the identical set of active 7-bit addresses: `0x27` (the PCF8574 LCD expander) and `0x68` (the DS3231 RTC). The physical protocol on the wire is completely agnostic of whether the master is an 8-bit AVR or a 32-bit ARM MCU; the slave ACKs whenever its unique 7-bit hardware comparator matches the broadcast address.

Task Question 2

Which address range does each platform scan, and why do we deliberately avoid `0x00` to `0x07` and `0x78` to `0x7F`?

Answer: Both platforms scan from 8 to 119 (`0x08` to `0x77`), providing 112 usable slave addresses. The excluded boundary ranges are strictly reserved under the official NXP UM10204 I^2C bus standard:

- `0x00` (0000000_2): **General Call Address** (used to broadcast to all slaves simultaneously) and **START**

byte (used by slow DSPs to synchronize clocks).

- `0x01` (0000001_2): **CBUS Address** (legacy protocol coexistence).
- `0x02` (0000010_2): Reserved for different bus formats.
- `0x03` (0000011_2): Reserved for future purposes.
- `0x04` to `0x07`: High-Speed mode (Hs-mode) master code.
- `0x78` to `0x7B` ($11110xx_2$): **10-bit Slave Addressing** prefix.
- `0x7C` to `0x7F` ($11111xx_2$): Reserved by NXP for device ID queries and future expansions.

Interrogating these reserved addresses during a routine scan can inadvertently reset devices (General Call) or trigger 10-bit addressing state machines on attached slaves, hanging the bus.

Task Question 3

Convert each detected address into its corresponding 8-bit write and read bytes.

Answer:

1. **PCF8574 LCD Backpack (7-bit address `0x27`):**

$$\begin{aligned} \text{Binary (7-bit):} &\quad 0100111_2 \\ \text{Write Byte } (R/\overline{W}=0)\text{:} &\quad (0x27 \ll 1) \mid 0 = 01001110_2 = \textbf{0x4E} \\ \text{Read Byte } (R/\overline{W}=1)\text{:} &\quad (0x27 \ll 1) \mid 1 = 01001111_2 = \textbf{0x4F} \end{aligned}$$

2. **DS3231 RTC Module (7-bit address `0x68`):**

$$\begin{aligned} \text{Binary (7-bit):} &\quad 1101000_2 \\ \text{Write Byte } (R/\overline{W}=0)\text{:} &\quad (0x68 \ll 1) \mid 0 = 11010000_2 = \textbf{0xD0} \\ \text{Read Byte } (R/\overline{W}=1)\text{:} &\quad (0x68 \ll 1) \mid 1 = 11010001_2 = \textbf{0xD1} \end{aligned}$$

5 Conclusion & Practical Engineering Takeaways

The dual-platform scanner confirms that both platforms achieve identical hardware address resolution. The primary engineering difference lies in software abstraction: Arduino's 'Wire' library accepts the raw 7-bit value and shifts it internally, whereas STM32's HAL requires the programmer to explicitly shift the 7-bit value ('addr « 1'), placing the responsibility for bit alignment on the embedded systems engineer.

Table 2: Bus Scanner Diagnostic Results Across Experimental Conditions

Experimental Condition	**Arduino Serial Output**	**STM32 UART2 Output**	**Physical / Electrical Explanation**
Baseline (LCD + DS3231 connected)	Found at `0x27` and `0x68`. Total: 2 devices.	Found at `0x27` and `0x68`. Total: 2 devices.	Both slaves recognize their address and pull SDA LOW on the 9th clock.
Fault Injection 1 (RTC Disconnected)	Found at `0x27`. Total: 1 device.	Found at `0x27`. Total: 1 device.	Address `0x68` receives no ACK; line floats HIGH via R_p (NACK). Scanner proceeds normally.
Fault Injection 2 (SDA Wire Disconnected)	"No I2C devices responded" (or hangs if internal timeouts not met).	No devices found; scanner completes immediately.	With SDA open, the master cannot sense any ACK pulse from any slave; all addresses read